

AEDD - Guía Práctica 14: Estructuras de datos dinámicas

Nota vinculada a los ejemplos de la presentación de este tema utilizada en teoría:

Dada la siguiente definición para una lista dinámica L:

```
struct Nodo {  
    int info;  
    struct Nodo * sig;  
};
```

```
typedef Nodo * ptrNodo ;
```

Una lista L se declara como un puntero a un nodo (el primero de la lista):

```
ptrNodo L;
```

Si L debe pasa como parámetro a una función por copia, en los casos en que el contenido de L sea cambiado durante la función, al retornar a la función llamadora se perderán esos cambios:

```
funcion1 ( ptrNodo L,.....)
```

Es decir que si L es un parámetro de entrada/salida, no podemos pasarlo por copia.

Para que los cambios en L se mantengan luego que la función finaliza hay 2 formas de realizarlo:

- a) Pasar L por referencia:

```
.....funcion1 ( ptrNodo & L,.....)
```
- b) Pasar un puntero a puntero que es la lista :

```
.....funcion1 ( ptrNodo * L,.....)
```

Ambas formas se suelen encontrar en la bibliografía y en códigos disponibles.

En la presentación de teoría se utilizaron ejemplos con la forma b), pero en esta práctica , a fin de continuar trabajando con pasaje por referencia , utilizaremos la forma a)

Ejercicio 1: Codificar las siguientes funciones:

1. Crear un nuevo nodo, reservando espacio en memoria.
2. Imprimir una lista. Ejemplo: 4 --> 5 --> 6 --> 7 --> 9 --> 78 --> 96 --> 98 --> NULL
3. Imprimir una lista pero en reverso.
4. Informar si una lista está vacía (o no).
5. Insertar un elemento al principio de una lista.
6. Insertar un elemento al final de una lista.
7. Borrar un elemento de una lista.
8. Borrar el primer nodo de la lista.
9. Insertar un elemento en una lista, en forma ordenada.

10. Ordenar una lista.

Ejercicio 2: Escribir una función que permita unir dos listas lineales A y B que contienen números enteros y devolver A U B (la lista resultado debe contener los elementos en A ó en B, sin repetidos).

Ejercicio 3: En una lista enlazada se guarda información números naturales de hasta 4 cifras.

La información se cargó en la lista de manera que esté ordenada en forma creciente. Pero se cree que hay algunos errores, es decir que hay tramos que no están en orden creciente.

Se debe construir una función que actualice la lista, eliminando los tramos que no están en forma creciente.

Ejemplo: 4 - 5 - 6 - ~~3~~ - ~~2~~ - ~~1~~ - 7 - 9 - 78 - ~~45~~ - ~~45~~ - ~~34~~ - 96 - 98 - ~~57~~

Ejercicio 4:

a) Declarar los tipos de datos necesarios para almacenar una lista secuencial enlazada cuyos nodos contienen la siguiente información:

- Una cadena de caracteres con el nombre de una ciudad.
- Las coordenadas X e Y de dicha ciudad en el plano.
- Un número de registración que se asigna aleatoriamente.

b) Escribir funciones que reciban la lista y permitan:

- Informar el nombre de la ciudad con el número de registro más bajo.
- Informar si la lista está ordenada alfabéticamente por nombre de ciudad.
- Eliminar de la lista todas las ciudades que tengan ordenada en el plano menor que 0.

Ejercicio 5: En dos listas Curso1 y Curso2 de tipo CALIFICACIONES se tiene la información de las notas obtenidas por los alumnos de dos comisiones de ISI, en las 6 asignaturas de primer año. Ambas listas están ordenadas por ID de los alumnos.

Se solicita integrar ambas listas generando una nueva.

Las listas de tipo CALIFICACIONES tienen en cada nodo la siguiente información: ID Alumno (6 cifras) y un vector con las 6 calificaciones finales de las asignaturas de primer año.

En la lista INTEGRADA deben aparecer los alumnos ordenados por promedio de las 6 calificaciones (de mayor a menor). La lista integrada contiene en cada nodo solamente el ID del alumno, y un valor real con el promedio obtenido.

Definir los tipos de datos y una función que reciba las dos listas y retorne la lista INTEGRADA.

Ejercicio 6: Codificar una función que reciba dos **listas enlazadas simples**, cuyos nodos contienen la siguiente información: NUM (un número) y LET (una letra); y **devuelva la primera** actualizada.

Ambas listas **están ordenadas** según el número de cada nodo.

Si en la primera lista hay algún nodo cuyo número coincide con el número de algún nodo de la segunda

lista, se reemplaza en el nodo de la primera lista el dato LET por el siguiente en el abecedario, y si es 'z' por 'a'.

PROBLEMAS INTEGRADOS

Ejercicio 1: Escribir la cabecera y cuerpo de una función **ejercicio1** que recibe una matriz de $N \times N$ números enteros y devuelve:

a) Una lista enlazada simple L, conteniendo los elementos de la matriz que representan valores “aislados” (aquellos que no pertenecen ni a la primera ni a la última columna, y que a su izquierda y derecha tienen valores distintos de él):

Ej.: Si M =	0	1	2	1	suma: 4
	1	1	0	1	suma: 3
	2	3	4	1	suma: 10
	4	4	7	0	suma: 15

la lista es $L \rightarrow 1 \rightarrow 2 \rightarrow 0 \rightarrow 3 \rightarrow 4 \rightarrow 7 \rightarrow \text{NULL}$

b) Una pila conteniendo el índice y la suma de cada fila, ordenada en forma ascendente.

la pila es: $P \rightarrow (2,3) \rightarrow (1,4) \rightarrow (3,10) \rightarrow (4,15) \rightarrow \text{NULL}$

Ejercicio 2: Codificar una función que recibe dos listas (LH, con datos de hombres, y LM, con datos de mujeres), cuyos nodos contienen la siguiente información:

```
struct datos { long dni, char apeynom[30], int edad };
```

Las listas están ordenadas en forma ascendente por el campo apeynom.

La función debe:

a) Generar una pila con la información de las 5 mujeres con menor dni, ordenada en forma descendente por dni (en la base de la pila quedará la persona con menor dni). Estos 5 registros deben eliminarse de las lista recibida.

b) Generar una cola con la información de las dos listas intercaladas (manteniendo el orden por apeynom), y agregando un campo con la información del sexo de cada persona.

c) Procesar la cola (completa ó los primeros 100 registros, lo que suceda primero): calcular la edad promedio de los hombres procesados.

Ejercicio 3:

Una imagen puede representarse como una matriz de puntos (pixels) donde para cada uno de ellos se debe indicar su tono de gris. Dicho tono es un número entre 0 y 255, donde 0 representa negro, 255 blanco y los valores intermedios dan el tono de gris correspondiente.

Como las imágenes ocupan un gran espacio de memoria, se pensó en usar una estructura como la siguiente:

[illegible]

en donde **Fila1**, **Fila2**, **FilaN** son punteros a los pixels de cada línea.

Estamos trabajando con un monitor que tiene una resolución de 480x640 pixels (o puntos). Se solicita que usted:

- 1- Defina la estructura de datos correspondiente.
- 2- Calcule el tamaño de la estructura al momento de comenzar el programa.
- 3- Escriba un módulo que le permita ingresar las primeras 100 filas de la pantalla (no olvide que cada fila está formada por 640 puntos). Indique el tamaño de la estructura una vez que el módulo se haya ejecutado.
- 4-Cuál es el tamaño de la estructura completa, es decir, cuando se hayan almacenado los 480 x 640 puntos.